

Lab 10: Natural Language Processing

Course: Programming for Artificial Intelligence

Instructor: Rasikh Ali

1. Introduction to the Task

Natural Language Processing (NLP) is a critical subfield of Artificial Intelligence that enables computers to process, understand, and generate human language. This project focuses on a core NLP application: **Text Classification**, specifically developed for **Spam Detection**.

The objective is to build a system capable of distinguishing between "Ham" (legitimate messages) and "Spam" (unsolicited/harmful messages) by analyzing patterns in word frequency and context.

Key Objectives:

- Implement text preprocessing techniques (Tokenization and Stopword removal).
- Convert raw text into numerical features using Vectorization.
- Train a Machine Learning classifier (Multinomial Naive Bayes).
- Deploy the model via a Flask-based web interface.

2. Technical Workflow

The development of the NLP Spam Classifier follows a structured pipeline:

1 Data Collection & Labeling

A dataset consisting of various messages is compiled. Each message is labeled as either 'spam' or 'ham'. This serves as the ground truth for the supervised learning model.

2 Text Preprocessing

Raw text is cleaned to reduce noise. This involves converting all text to lowercase and removing "Stopwords" (e.g., 'is', 'the', 'and') which do not contribute to the semantic classification of the message.

3 Feature Extraction (Vectorization)

Computers cannot process words directly. We use the **Bag of Words (BoW)** model via *CountVectorizer*. This creates a matrix where each row represents a message and each column represents a unique word from the vocabulary. The values represent word frequencies.

4 Model Training

The *Multinomial Naive Bayes* algorithm is applied. It calculates the probability of a message belonging to a certain class based on the words it contains, using Bayes' Theorem:

$$P(\text{Label} | \text{Word}) = (P(\text{Word} | \text{Label}) \times P(\text{Label})) / P(\text{Word})$$

5 Model Serialization

Once trained, the model and the vectorizer are saved as "Pickle" (.pkl) files. This allows the web server to load the intelligence without needing to retrain the model every time a user submits a request.

3. Frontend Design & User Interaction

To make the system accessible, a **Flask Web Application** is implemented. The frontend (HTML/CSS) provides a simple interface where users can type or paste a message.

When the "Predict" button is clicked, the following happens:

1. The message is sent to the Flask backend via a POST request.
2. The backend loads the stored model and transforms the user input into the correct numerical format.
3. The model makes a prediction.
4. The result is displayed on the screen with visual indicators (e.g., Red for Spam, Green for Ham).

4. Conclusion

This lab task demonstrates the full lifecycle of an AI application, from raw data processing to web deployment. By utilizing NLP concepts like tokenization and probabilistic classification, we can automate the detection of malicious content with high accuracy.